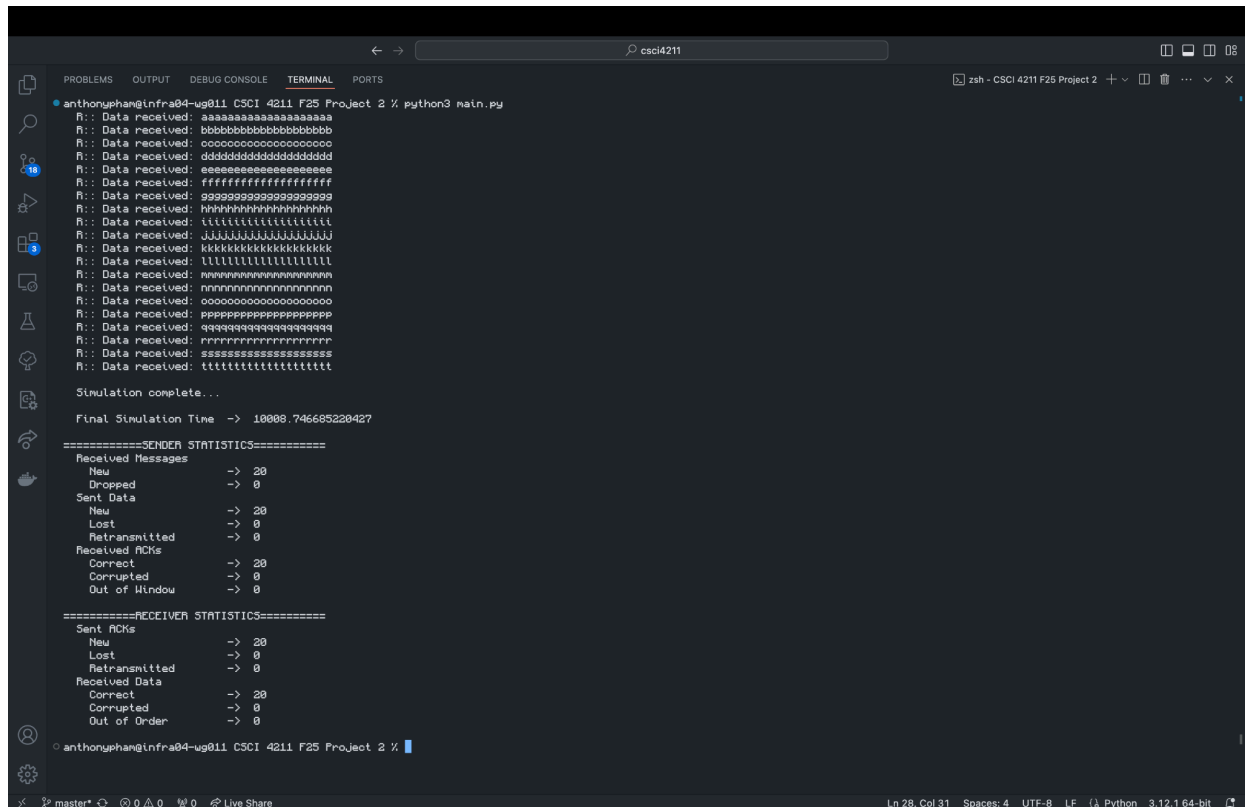


Anthony Pham
Dr. Eman Ramadan
CSCI 4211
14 November 2025

Project 2 Report

Test Case Analysis

Test case 1: nsimmax = 20, lossprob = 0, corruptprob = 0, Lambda = 500



```
anthonypham@lnfra04-ug011: CSCI 4211 F25 Project 2 % python3 main.py
A:: Data received: aaaaaaaaaaaaaaaaaa
R:: Data received: bbbbbbbbbbbbbbbbbbbb
R:: Data received: cccccccccccccccccccc
R:: Data received: dddddddddddddddddddd
R:: Data received: eeeeeeeeeeeeeeeeeeee
R:: Data received: ffffffffffffffffffff
R:: Data received: gggggggggggggggggggg
R:: Data received: hhhhhhhhhhhhhhhhhhhh
R:: Data received: iiiiiiiiiiiiiiiiiiii
R:: Data received: jjjjjjjjjjjjjjjjjjjj
R:: Data received: kkkkkkkkkkkkkkkkkkkk
R:: Data received: llllllllllllllllllll
R:: Data received: mmmmmmmmmmmmmmmmmmm
R:: Data received: nnnnnnnnnnnnnnnnnnnn
R:: Data received: oooooooooooooooooooo
R:: Data received: pppppppppppppppppppp
R:: Data received: qqqqqqqqqqqqqqqqqqqq
R:: Data received: rrrrrrrrrrrrrrrrrrrr
R:: Data received: ssssssssssssssssssss
R:: Data received: tttttttttttttttttttt

Simulation complete...

Final Simulation Time -> 10008.746685220427

=====SENDER STATISTICS=====
Received Messages
  New      -> 20
  Dropped  -> 0
Sent Data
  New      -> 20
  Lost     -> 0
  Retransmitted -> 0
Received ACKs
  Correct  -> 20
  Corrupted -> 0
  Out of Window -> 0

=====RECEIVER STATISTICS=====
Sent ACKs
  New      -> 20
  Lost     -> 0
  Retransmitted -> 0
Received Data
  Correct  -> 20
  Corrupted -> 0
  Out of Order -> 0

anthonypham@lnfra04-ug011: CSCI 4211 F25 Project 2 %
```

Test case 2: nsimmax = 20, lossprob = 0.2, corruptprob = 0, Lambda = 500

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
anthonyphang@infra04-ug011 CSCI 4211 F25 Project 2 % python3 main.py
R:: Data received: aaaaaaaaaaaaaaaaaa
R:: Data received: bbbbbbbbbbbbbbbbbb
R:: Data received: cccccccccccccccccc
R:: Data received: dddddddddddddddddd
R:: Data received: eeeeeeeeeeeeeeeeee
R:: Data received: ffffffffffffffffffff
R:: Data received: gggggggggggggggggggg
R:: Data received: hhhhhhhhhhhhhhhhhhhh
R:: Data received: iiiiiiiiiiiiiiiiiiii
R:: Data received: jjjjjjjjjjjjjjjjjjjj
R:: Data received: kkkkkkkkkkkkkkkkkkkk
R:: Data received: llllllllllllllllllll
R:: Data received: mmmmmmmmmmmmmmmmmm
R:: Data received: nnnnnnnnnnnnnnnnnnnn
R:: Data received: oooooooooooooooooooo
R:: Data received: ppppppppppppppppppppp
R:: Data received: qqqqqqqqqqqqqqqqqqqq
R:: Data received: rrrrrrrrrrrrrrrrrrrr
R:: Data received: ssssssssssssssssssss
R:: Data received: tttttttttttttttttttt

Simulation complete...

Final Simulation Time -> 10011.72036745696

=====SENDER STATISTICS=====
Received Messages
  New      -> 20
  Dropped  -> 0
Sent Data
  New      -> 20
  Lost     -> 10
  Retransmitted -> 17
Received ACKs
  Correct  -> 20
  Corrupted -> 0
  Out of Window -> 0

=====RECEIVER STATISTICS=====
Sent ACKs
  New      -> 20
  Lost     -> 7
  Retransmitted -> 0
Received Data
  Correct  -> 20
  Corrupted -> 0
  Out of Order -> 7

anthonyphang@infra04-ug011 CSCI 4211 F25 Project 2 %
```

Test case 3: nsimmax = 20, lossprob = 0, corruptprob = 0.3, Lambda = 500

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
anthonyphang@infra04-ug011 CSCI 4211 F25 Project 2 % python3 main.py
R:: Data received: aaaaaaaaaaaaaaaaaa
R:: Data received: bbbbbbbbbbbbbbbbbb
R:: Data received: cccccccccccccccccc
R:: Data received: dddddddddddddddddd
R:: Data received: eeeeeeeeeeeeeeeeee
R:: Data received: ffffffffffffffffffff
R:: Data received: gggggggggggggggggggg
R:: Data received: hhhhhhhhhhhhhhhhhhhh
R:: Data received: iiiiiiiiiiiiiiiiii
R:: Data received: jjjjjjjjjjjjjjjjjjjj
R:: Data received: kkkkkkkkkkkkkkkkkkkk
R:: Data received: llllllllllllllllllll
R:: Data received: mmmmmmmmmmmmmmmmmm
R:: Data received: nnnnnnnnnnnnnnnnnnnn
R:: Data received: oooooooooooooooooooo
R:: Data received: ppppppppppppppppppppp
R:: Data received: qqqqqqqqqqqqqqqqqqqq
R:: Data received: rrrrrrrrrrrrrrrrrrrr
R:: Data received: ssssssssssssssssssss
R:: Data received: tttttttttttttttttttt

Simulation complete...

Final Simulation Time -> 10010.582736418162

=====SENDER STATISTICS=====
Received Messages
New          -> 20
Dropped      -> 0
Sent Data
New          -> 20
Lost         -> 0
Retransmitted -> 13
Received ACKs
Correct      -> 20
Corrupted   -> 9
Out of Window -> 4

=====RECEIVER STATISTICS=====
Sent ACKs
New          -> 20
Lost         -> 0
Retransmitted -> 11
Received Data
Correct      -> 20
Corrupted   -> 11
Out of Order -> 2

anthonyphang@infra04-ug011 CSCI 4211 F25 Project 2 %
```

Test case 4: nsimmax = 20, lossprob = 0.2, corruptprob = 0.3, Lambda = 500

```
cscl4211
zsh - CSCI 4211 F25 Project 2
anthonyphang@infra04-ug011 CSCI 4211 F25 Project 2 % python3 main.py
R:: Data received: aaaaaaaaaaaaaaaaaa
R:: Data received: bbbbbbbbbbbbbbbbbb
R:: Data received: cccccccccccccccccc
R:: Data received: dddddddddddddddddd
R:: Data received: eeeeeeeeeeeeeeeeee
R:: Data received: ffffffffffffffffffff
R:: Data received: gggggggggggggggggggg
R:: Data received: hhhhhhhhhhhhhhhhhhhh
R:: Data received: iiiiiiiiiiiiiiiiiiii
R:: Data received: jjjjjjjjjjjjjjjjjjjj
R:: Data received: kkkkkkkkkkkkkkkkkkkk
R:: Data received: llllllllllllllllllll
R:: Data received: mmmmmmmmmmmmmmmmmm
R:: Data received: nnnnnnnnnnnnnnnnnnnn
R:: Data received: oooooooooooooooooooo
R:: Data received: ppppppppppppppppppppp
R:: Data received: qqqqqqqqqqqqqqqqqqqq
R:: Data received: rrrrrrrrrrrrrrrrrrrr
R:: Data received: ssssssssssssssssssss
R:: Data received: tttttttttttttttttttt

Simulation complete...

Final Simulation Time -> 10009.86202209262

=====SENDER STATISTICS=====
Received Messages
  New      -> 20
  Dropped  -> 0
Sent Data
  New      -> 20
  Lost     -> 5
  Retransmitted -> 23
Received ACKs
  Correct  -> 20
  Corrupted -> 4
  Out of Window -> 3

=====RECEIVER STATISTICS=====
Sent ACKs
  New      -> 20
  Lost     -> 11
  Retransmitted -> 9
Received Data
  Correct  -> 20
  Corrupted -> 9
  Out of Order -> 9

anthonyphang@infra04-ug011 CSCI 4211 F25 Project 2 %
```

Test case 5: nsimmax = 20, lossprob = 0.8, corruptprob = 0.8, Lambda = 250

```
cscl4211
zsh - CSCI 4211 F25 Project 2 + - 08
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
anthonyphang@infra04-ug011: CSCI 4211 F25 Project 2 % python3 main.py
R:: Data received: aaaaaaaaaaaaaaaaaa
R:: Data received: bbbbbbbbbbbbbbbbbb
R:: Data received: cccccccccccccccccc
R:: Data received: dddddddddddddddddd
R:: Data received: eeeeeeeeeeeeeeeeee
R:: Data received: ffffffffffffffffffff
R:: Data received: gggggggggggggggggggg
R:: Data received: hhhhhhhhhhhhhhhhhh
R:: Data received: jjjjjjjjjjjjjjjjjjjj

Simulation complete...

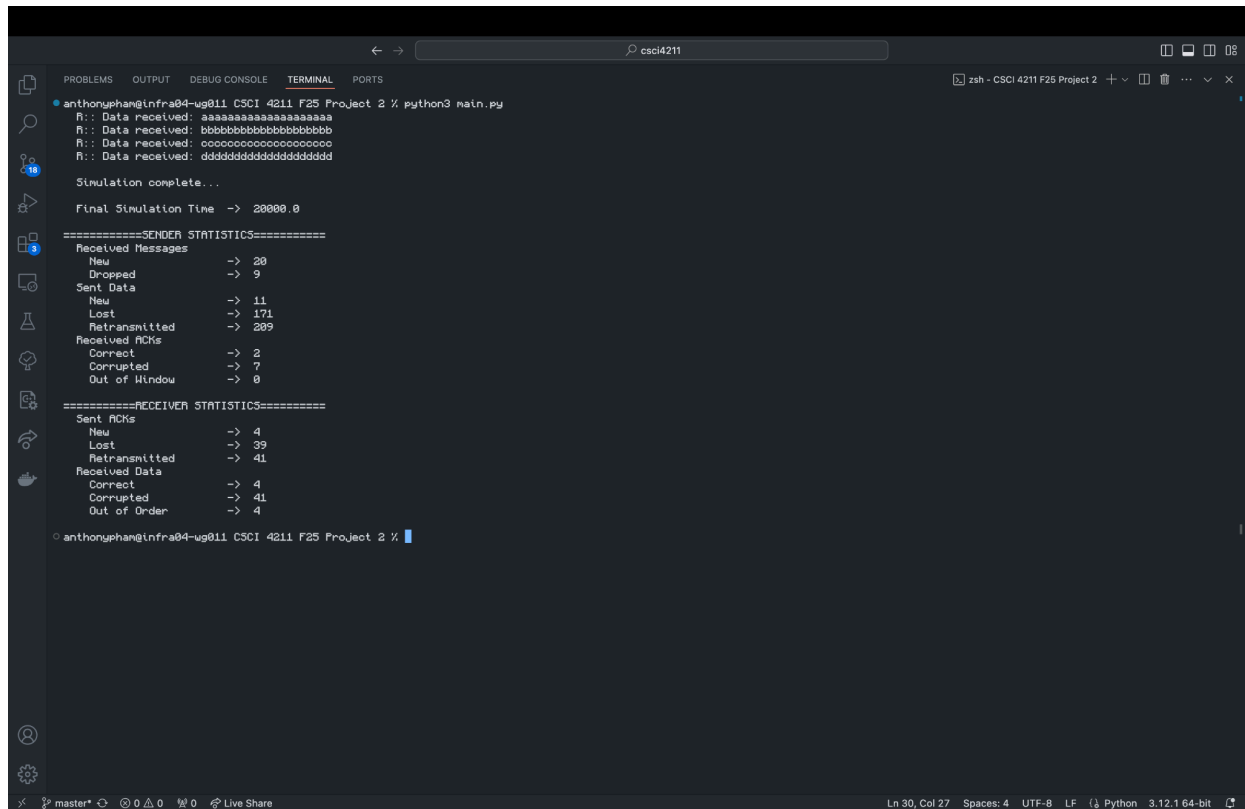
Final Simulation Time -> 5247.87489887464

=====SENDER STATISTICS=====
Received Messages
New -> 20
Dropped -> 5
Sent Data
New -> 15
Lost -> 722
Retransmitted -> 896
Received ACKs
Correct -> 1
Corrupted -> 28
Out of Window -> 3

=====RECEIVER STATISTICS=====
Sent ACKs
New -> 9
Lost -> 154
Retransmitted -> 153
Received Data
Correct -> 9
Corrupted -> 153
Out of Order -> 27

anthonyphang@infra04-ug011: CSCI 4211 F25 Project 2 %
```

Test case 6: nsimmax = 20, lossprob = 0.8, corruptprob = 0.8, Lambda = 1000



```
anthonyphan@infra04-ug011: CSCI 4211 F25 Project 2 % python3 main.py
R:: Data received: aaaaaaaaaaaaaaaaaa
R:: Data received: bbbbbbbbbbbbbbbbbb
R:: Data received: cccccccccccccccccc
R:: Data received: dddddddddddddddddd

Simulation complete...

Final Simulation Time -> 20000.0

=====SENDER STATISTICS=====
Received Messages
  New      -> 20
  Dropped  -> 9
Sent Data
  New      -> 11
  Lost     -> 171
  Retransmitted -> 209
Received ACKs
  Correct  -> 2
  Corrupted -> 7
  Out of Window -> 0

=====RECEIVER STATISTICS=====
Sent ACKs
  New      -> 4
  Lost     -> 39
  Retransmitted -> 41
Received Data
  Correct  -> 4
  Corrupted -> 41
  Out of Order -> 4

anthonyphan@infra04-ug011: CSCI 4211 F25 Project 2 %
```

Test case 1 acted as the control here. All 20 packets were sent and received with no losses or retransmissions. This should be expected, since nothing got lost or corrupted, and no retransmissions nor mismatched arrival timings have occurred as a result, simulating a channel in ideal conditions.

Once a loss probability of 20 percent was added, things started getting a little shaky. The sender's messages were lost along the way 10 times, and the receiver's ACKs were lost 7 times. These contributed to the 17 retransmissions from the sender.

When going with no loss probability and 30% corruption probability, the sender and receiver both had to retransmit their packets and experienced out-of-order arrivals and dropped packets from a full window. With the corruption, both the sender and receiver had packets with mismatched checksums, resulting in retransmissions. Additionally, arrival timing from both sides

must have gotten mismatched as a result of the retransmissions, causing the out-of-order packets and window saturation.

Test case 4, with a loss probability of 20% and a corruption probability of 30%, showed increased retransmissions from the sender as a result of a combination of the receiver sending retransmitted ACKs from receiving corrupted packets and from packets being lost during transmission.

Test case 5 is where things got a lot more messy. Having both loss and corruption probability at 80% meant that a majority of packets will have problems along the way. The increased rate of messages received by the sender as a result of Lambda being halved meant that packets could not be transmitted and ACK'd in time for the new messages to be added to the window, resulting in the window barely being able to clear and packet drops occurring. All of these factors explain the sharp increase in packet losses, retransmissions, and out-of-order packets.

The last test case had similar settings to test case 5, except Lambda was increased by a factor of four. This meant that the sender was receiving Application Layer messages at a fourth of the rate of the last test case, which reduces the pressure of adding packets to the window and timeouts. However, the results here showed a loss amount that was almost double the amount as the last test case. This could have been an odd case, and after running the test case multiple times, the results seemed to vary wildly. Either dropped packets or lost and retransmitted were extremely high, like as shown in the screenshot, or very low (with dropped packets often being zero). Additionally, the severity of these rates seemed to be inversely related. When dropped packets were low, loss and corruption were at their extremes, and vice versa.

Differences Between Go-Back-N and Selective Repeat

The key difference between Go-Back-N and Selective Repeat is that Selective Repeat's receiver has a buffer for its out-of-order packets. This means that the receiver does not have to strictly receive only the one in-order packet and sends the ACKs for all the packets it receives in its window. Because each packet in the receiver's window, out of order or not, will have their ACKs sent to the sender, the sender can see which packets were received and retransmit only the packets that weren't. This reduces the total amount of retransmissions that need to be done down to the packets that actually need retransmission.

Additionally, the window sizes for both the sender and receiver are different in SR. The receiver can have a window greater than one compared to GBN, but both sender and receiver windows must be less than or equal to half the sequence number space.

Modifications to Implement Selective Repeat

One of the steps to modify Go-Back-N to work for Selective Repeat is to change the window size for the receiver. The receiver (as well as the sender) should have a window size greater than or equal $S/2$. The receiver should also have an array or buffer to track which packets within the window have been received and had their ACKs sent. It should send an ACK for each packet received in the window, instead of just the ACK for the one in-order packet. Then the receiver should send the packets up to the Application Layer if and only if the lowest missing packet in the window gets received.

The sender also needs a way to track the state of each packet in its window. For each packet that gets ACKed in the sender's window, the sender should be able to store the packet's ACKed state, perhaps through an *isAked[N]* array. Also, each packet should have its own

individual timer which starts when the packet is sent and stops when the packet is ACKed. In SR, the window should only slide when the lowest unACKed packet is acknowledged, so the base of the sender's window should only be updated when that acknowledgement happens.

Because SR utilizes packet buffering in its receiver side and individual acknowledgement handling, the window sizes in SR differ from GBN. Like GBN, SR would implement rotating sequence numbers, but because the receiver's window is larger and received packets and ACKs are buffered, the window sizes should be $\leq S/2$ and both windows should match. This is to prevent ambiguity between a received sequence number within the window and outside the window. Having matching window sizes ensures that packets aren't unnecessarily dropped when one window size is smaller.